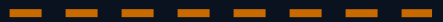
OBJECT-ORIENTED ANALYSIS & DESIGN

Association & Dependency



association



dependency

One line changed. Five classes broke.

TaxCalculator.java — Monday

```
// Monday, shipping code
public final class TaxCalculator {
  public static BigDecimal tax(
          BigDecimal net) { … }
}
```

TaxCalculator.java — Tuesday

```
// Tuesday: VAT now depends on country
public final class TaxCalculator {
  public static BigDecimal tax(
          BigDecimal net, Country c) { … }
}
```

Wednesday morning, the build server says:

✗ Receipt.java ✗ Order.java ✗ Invoice.java ✗ MonthlyReport.java ✗ CheckoutService.java

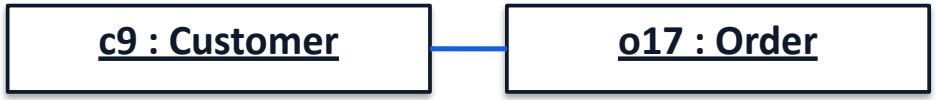
Nobody “touched” those five files. So why did they break?

Because each of them was already tied to TaxCalculator's definition, the compiler simply made the tie visible. UML gives that tie a name and a glyph: a dependency. And it gives the other tie, objects holding on to objects, a second glyph: an association.

Two questions. Two lines.

QUESTION 1 · STRUCTURE

“Which object is wired to which, while the program runs?”



o17 keeps a reference to c9 in a field. That stored reference is structure: it outlives the method call. UML draws it as a **solid line**, an association (between the objects: a link).

QUESTION 2 · CHANGE

“If I edit class B, which classes must I re-check?”



Receipt calls TaxCalculator's method, so Receipt's correctness rests on TaxCalculator's definition. UML draws it as a **dashed arrow**, a dependency, client to supplier.

Association the structural line

What the UML specification actually says, how to spot an association in code, and how to adorn the line so it carries real information.

What is an Association?

Real-World Example

Person & Wallet

A Person carries a Wallet in their pocket as a permanent field ().

```
private Wallet myWallet;
```

Key Explanation

Class A **stores** a reference to Class B.

The Wallet stays saved with the Person long-term. Drawn as a **solid line** ().

```
——————
```

Association vs. Link

CLASS LEVEL

Association

Student ——— School

The general blueprint rule: "Students attend Schools."

OBJECT LEVEL

Link

Alex ——— Springfield High

The specific connection running right now in memory.

An association is between classes. The thing between objects has its own name: a link.

UML 2.5.1, in plain words

An association is a relationship between classifiers (classes, interfaces…) stating that their instances can be linked.

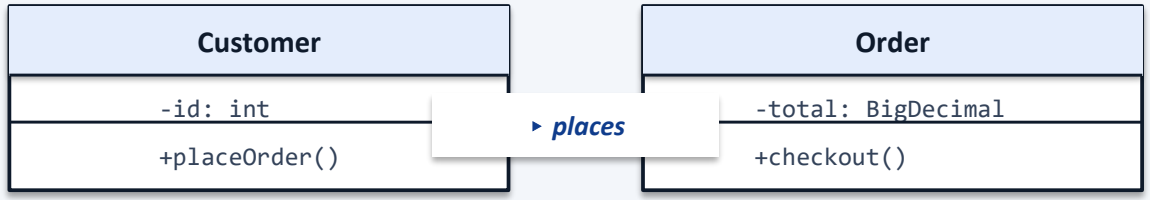
A link is one instance of an association: a tuple with exactly one object per end of the line.

So: one line on the diagram → potentially millions of links at runtime.

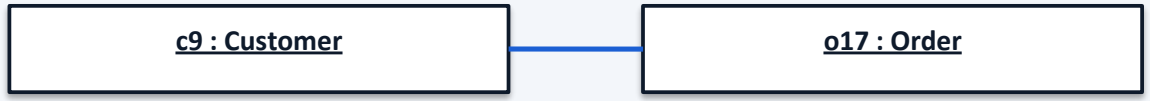
The correction that saves marks

Association is a class-level (design-time) relationship. Saying “association is between objects” confuses the line with one of its instances, that instance is the link.

SAME SOLID LINE, TWO LEVELS



class level: one association, Customer–Order



object level: one link, an instance of that association

Underlined names = object instances (UML instance notation). The link draws with the same solid line; only the boxes change.Ask yourself both levels every time you draw: “what does the class diagram promise, and what does one running object actually hold?”

the whole example

```
class Student {
    Course myCourse;
}
```



one link, while the program runs

How do you know an association exists? Visibility the four ways A can reach B

For an object of A to send a message to an object of B, B must be **visible** to A. Larman's GRASP “Visibility” pattern lists exactly four ways.

#	Visibility	Meaning	Java trace
1	Attribute	B is a field of A, the reference is stored and outlives the call	private Cart cart;
2	Parameter	B arrives as an argument of A's method	void apply(PromoCode c)
3	Local	B is a non-parameter local inside A's method, often created there	TaxCalculator t = new TaxCalculator();
4	Global	B is reachable from anywhere: static utility, singleton, module	PriceEngine.total(cart)

Source: C. Larman, Applying UML and Patterns, 3rd ed.: GRASP: Visibility. The same four appear in Fowler's UML Distilled as “how does A know about B?”

Printer meets Document in four different ways

```
class Printer {
    Document saved;                    // 1 field
    void print(Document d) {           // 2 parameter
        Document temp = new Document(); // 3 local
        Logger.log("done");            // 4 global
    }
}
```

- For Printer to send a message to a Document, it must be able to **see** one. There are only four ways:
  - keep it in a field (saved)
  - receive it as a parameter (d)
  - make it as a local variable (temp)
  - use one that everyone can see (Logger)

Sorting visibility

STORED → ASSOCIATION

Attribute visibility. The field keeps the reference alive after the method returns, so the structure is part of the object's state. Draw the solid line with multiplicity and navigability.

USED-ONLY → DEPENDENCY

Parameter, local, global visibility. The reference dies when the call returns. Draw the dashed arrow.

THE EXCEPTION

A parameter you keep. Assign the parameter into a field and parameter visibility becomes attribute visibility (Larman says this transformation is common). The relationship upgrades to an association at that line of code.

the upgrade happens here

```
public void assign(Rider r) {
  this.rider = r;    // parameter → field
}
```

and here it never does

```
public BigDecimal total(PromoCode c) {
```

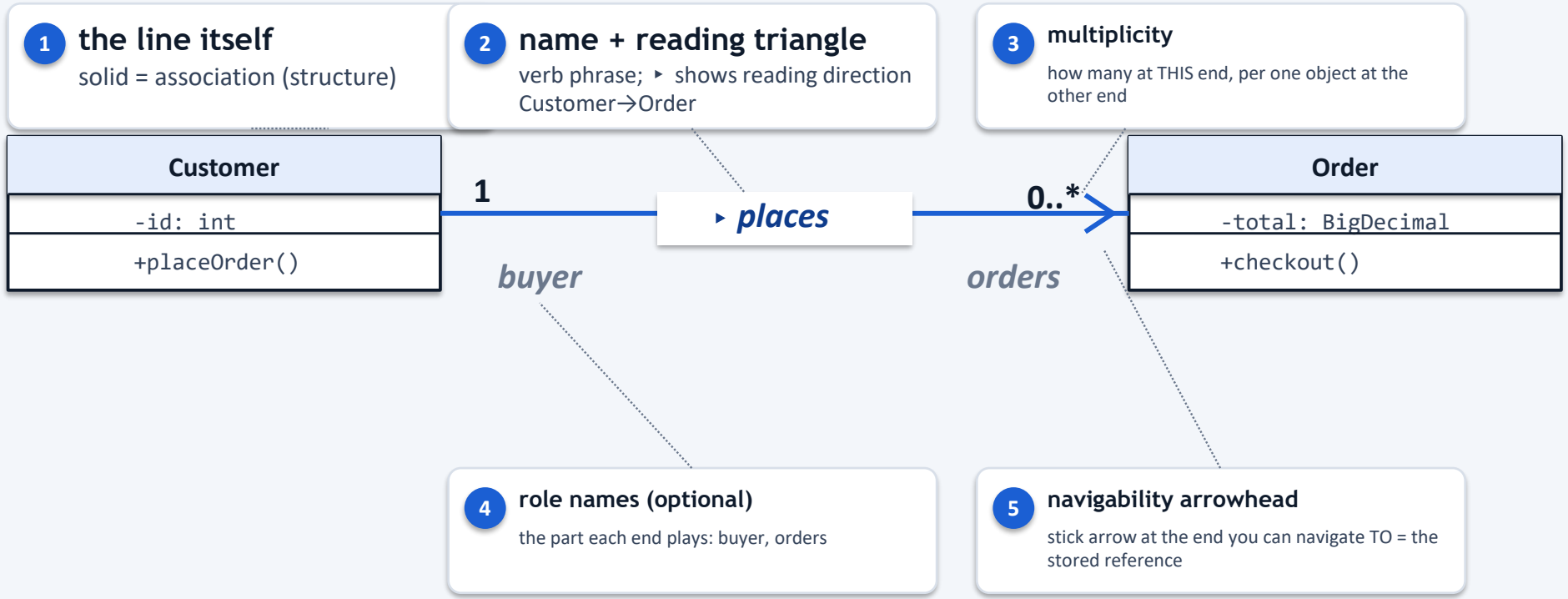
Stored ⇒ association. Used-only ⇒ dependency.

one class, two different relationships

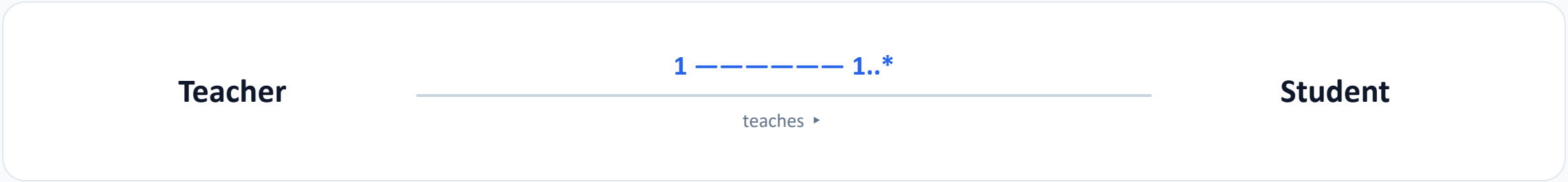
```
class Customer {
    Receipt myReceipt;        // stored

    void show(Receipt r) {
        r.print();            // used once
    }
}
```

Anatomy of the line: five decorations, each carrying one fact



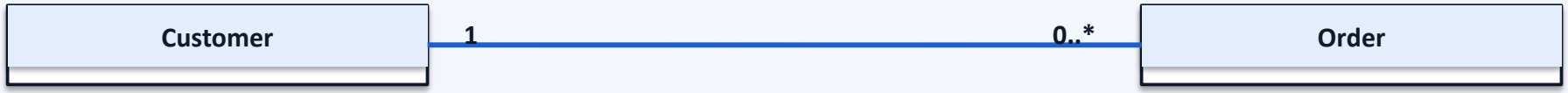
Understanding Multiplicity



How to Read: Stand at **Teacher** and look opposite: 1 Teacher teaches **1 or more (1..*)** Students. Stand at **Student** and look opposite: Each Student has exactly **1** Teacher.

Multiplicity

Rule. The multiplicity written at an end answers: how many objects of THIS end's class may be linked to ONE object of the class at the OTHER end? You always read it standing at the opposite end.



standing at Order, looking left: every Order is placed by exactly **1** Customer.

standing at Customer, looking right: every Customer has placed **0 or more** Orders

THE CLASSIC MISREAD (from an older deck)

Student “1” — “0..*” Course was explained as “exactly 1 student enrolls in 0 or more courses”. Backwards.

Read properly: each Course has exactly 1 Student, each Student takes 0..* Courses, which also exposes a suspect model: real courses have many students, so the multiplicities were probably meant the other way round.

THE NOTATION IS RICHER THAN YOU NEED

Legal values: exact (3), range (2..15), star (*), and even sets (3, 12..15, 901..*).

Sets are legal UML but in a business model they almost always mean “somebody has not decided”. Prefer 0..1, 1, 0..*, 1..*, n..m.

Multiplicity

Fragment	Verdict & reason
`Order "1" –contains▸– "1..*" OrderLine`	✓ each order has at least one line (an empty order is not an order); each line lives in exactly one order.
`Customer "1" –places– "1..*" Order`	✗ a registered customer with no orders yet must exist → 0..* at the Order end.
`OrderLine "1..*" –refers▸– "1" MenuItem`	✗ half wrong: a MenuItem may appear in zero lines → 0..* at the OrderLine end; the 1 at MenuItem is right.
`Restaurant "1" –offers▸– "0..*" MenuItem`	✓ every menu item belongs to one restaurant; a restaurant may list none yet.
`Rider "0..1" –delivers– "0..*" Order`	✓ an order may await a rider (0..1 at Rider end); a rider accumulates many orders over a shift.
`Store "1" – "1, 5..10, 50..*" Product`	✗ legal syntax, meaningless rule — size of store is not a property of one product link. Model it differently.

Say the sentence out loud in the "each X … one Y" form; if you cannot, the multiplicities are in the wrong seats.

Navigability: the arrowhead answers “who holds the reference?”



unspecified

No adornment: the model does not commit. Tools draw this by default; in code terms, anything is still possible.

```
// decision deferred
```



navigable A → B

Stick arrowhead at B: from an A you can efficiently reach its B. In code: A stores a field of type B.

```
class A { private B b; }
```



navigable A → B, and B → A forbidden

Add a small × at the A end: B objects cannot navigate back. A stronger claim, same code, stricter design intent.

```
class A { private B b; }  // B holds nothing
```

Arrowhead points at the class whose reference is stored, the destination of navigation, not the direction of any message at runtime.

Navigability: Who Holds Whom?

Customer ——————> Order

Arrow points to destination

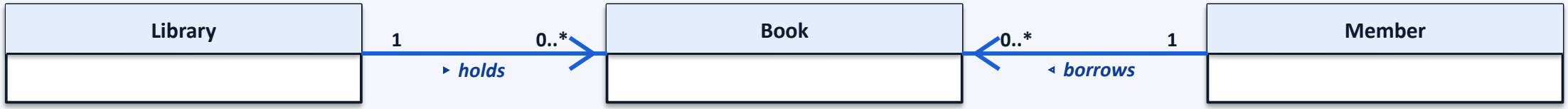
Customer holds a list of **Orders** inside a field. Customer can easily look up its Orders.

Customer <—————— Order

Reversed direction

Order holds a reference to **Customer**. An Order knows who bought it, but Customer stores no order list.

A whole model in three classes

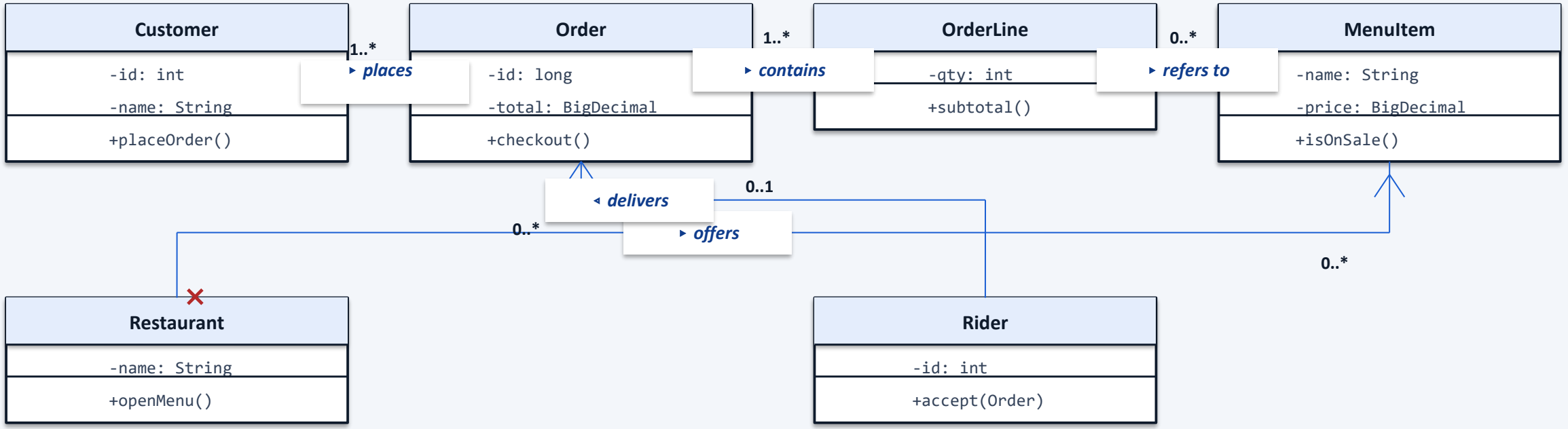


IN PLAIN WORDS

- Three classes, two solid lines. Each line has a verb and two numbers. That is a complete structural model.
- Read line 1: each book is held by 1 library; each library holds 0 or more books; the library keeps the list (arrowhead at Book).
- Read line 2: each borrowed book is borrowed by 1 member; each member borrows 0 or more books; the member keeps the list (arrowhead at Book from the right).

Remember: a model is just classes + solid lines + verbs + numbers: nothing else is required.

Running example: the CampusEats structural model (associations only)



Read one line aloud as a model: **`Order "1" —contains▸— "1..*" OrderLine`** means: standing at an OrderLine, exactly one Order; standing at an Order, one or more lines; the arrowhead says Order keeps the list, not the other way round. The × on the Restaurant end says a MenuItem cannot walk back to its restaurant, a deliberate design decision, not an omission.

Dependency, the change line

The specification's definition, the five places it hides in code, and the theorem that ties it to association.

What is a Dependency?

🍴 Real-World Example

Chef & Recipe Book

A Chef reads a Recipe Book while cooking a meal. Once finished, the Chef puts the book away.

Key Explanation

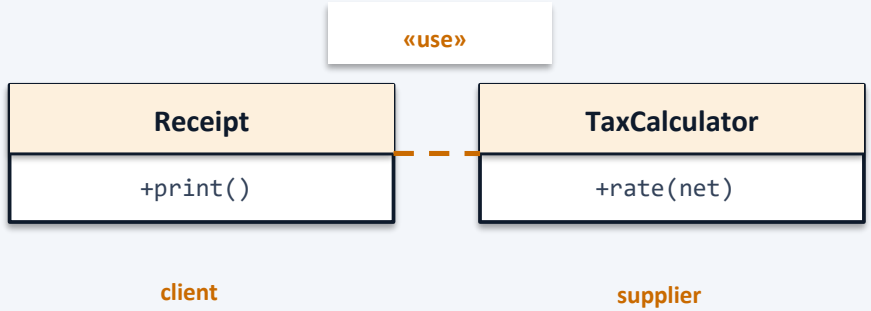
Class A **uses** Class B temporarily.

If the Recipe Book changes its steps, the Chef's cooking changes. Drawn as a **dashed arrow** ().

`- - - >`

The specification's sentence, unpacked

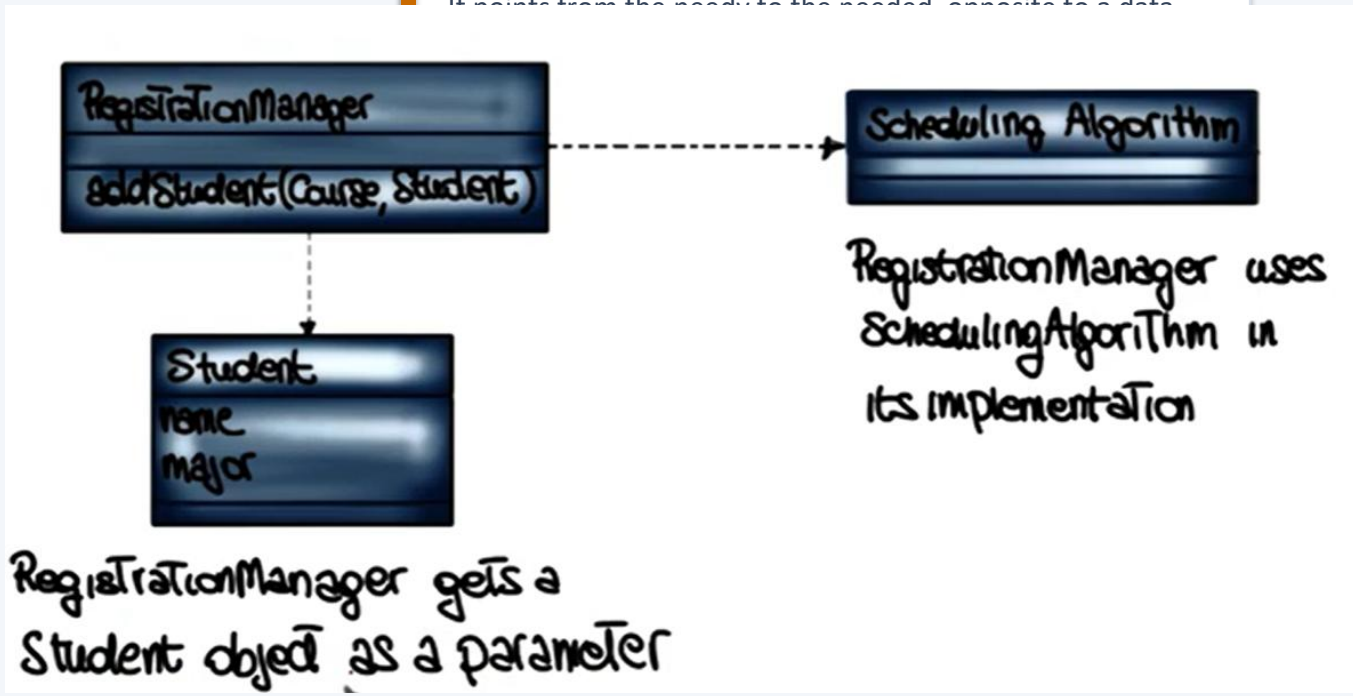
> *"A Dependency is a Relationship that signifies that a single model Element or a set of model Elements requires other model Elements for their specification or implementation."*
>
> UML 2.5.1 (OMG formal/2017-12-05), clause on Dependencies



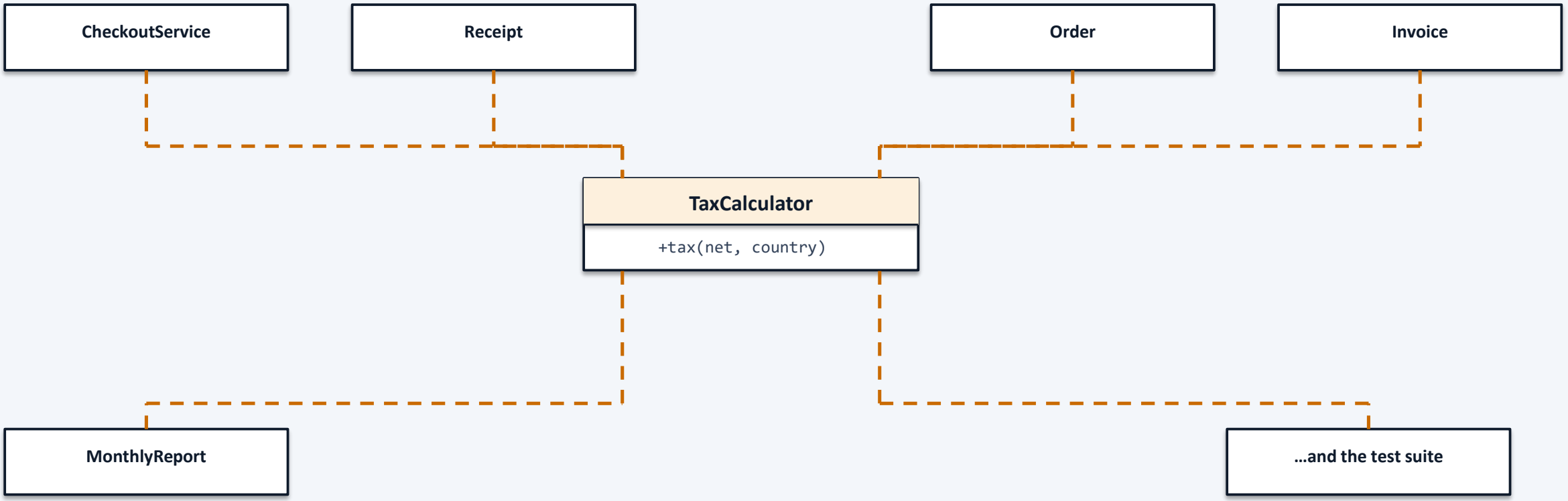
- **Client → supplier.** The arrow leaves the dependent (client) and points at the required (supplier). Dashed line, open arrowhead.
- **Operational test:** "if B's definition changes, must A be re-examined or recompiled?" Yes ⇒ A depends on B.
- It relates **definitions**, classes, interfaces, packages, not running objects.

Note the honesty of the definition: it says ***may*** require. A dependency is a warning shot, not a prophecy: change the supplier in a way the client never uses, and the client survives.

Why the arrow feels "backwards"



Example



Model-time, not run-time: a dependency is a fact about definitions. Two objects need never meet for it to be true.

...but it bites at run-time too: ship without the supplier and the JVM throws NoClassDefFoundError; Python raises ImportError. Design debt, runtime interest.

Five code signals that create a dependency

#	Signal in the code	Java example	Stereotype
1	Parameter type of a method	void apply(PromoCode c)	«use»
2	Local variable / object you create	TaxCalculator t = new TaxCalculator();	«create» / «instantiate»
3	Static or module-level call	PriceEngine.total(cart)	«call»
4	Return type of a method	Receipt makeReceipt()	«use»
5	A stored field	private Customer buyer;	Association, which implies the dependency (never draw both)

the same five signals in Python

```python
import pricing            # module dependency
from promo import PromoCode

def checkout(cart, code):     # parameter
    tax = pricing.TaxCalculator()   # local+create
    return pricing.total(cart, tax) # static call
```

Homework.java

```java
class Homework {
    Course course;                  // field
    void check(Answer a) { }        // parameter
    void make() {
        Sheet s = new Sheet();      // local, created
        int n = Helper.add(1, 2);   // static call
    }
    Answer get() { return null; }   // return type
}
```

Every association is also a dependency

DEPENDENCY · A needs B's definition

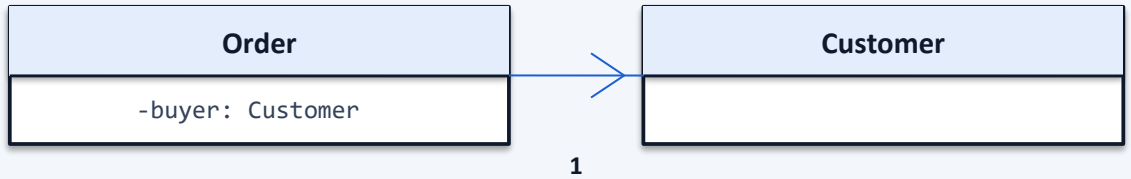
ASSOCIATION · A stores B

a field of type B means the compiler needs B to compile A, the definition of a dependency.

…but not every dependency is an association: parameters, locals and static calls leave no stored reference behind.

one field

```
class Order {
    Customer customer;
}
```





redundant: do not draw the dashed arrow next to its own association.

Association ⇒ dependency. Dependency ⇏ association. *Draw the stronger fact; let the weaker one stay implicit.*

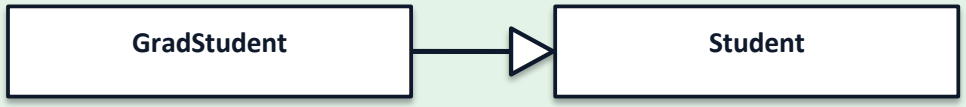
Inheritance is not a dependency arrow (and three arrowheads you must not swap)

THE OLD SLIDE'S EXAMPLE



Yes, changing Student affects GradStudent — but the relationship is generalization and has its own glyph. Drawing a dashed arrow here is a notation error.

WHAT YOU DRAW INSTEAD



Solid line, hollow triangle at the parent. The coupling is real and stronger than any dependency — it simply is not this glyph.

THE ARROWHEAD ALPHABET — FOUR MARKS, FOUR MEANINGS



stick arrowhead on a solid line
= navigable association end

open arrowhead on a dashed line
= dependency, client → supplier



hollow triangle on a solid line
= generalization (inheritance)



small × on a solid line
= non-navigable end

Exam economics: swapping any two of these costs you the whole relationship. When in doubt, say the meaning out loud first (“stores”, “needs”, “is-a”, “cannot navigate”) and then pick the glyph.

The stereotype gallery: what UML actually standardises

YOU WILL DRAW THESE ON CLASS DIAGRAMS

«use»

generic usage; the default reading of a bare dashed arrow on a class diagram

«create» / «instantiate»

client's operations create instances of the supplier

«call»

one operation invokes another (operation → operation, or class → class by extension)

REAL, BUT THEY LIVE ELSEWHERE

«import» / «access» / «merge»

namespace & package level: what a package makes visible to whom

«send»

an operation emits a signal — lives on interaction diagrams

«trace» / «refine» / «derive»

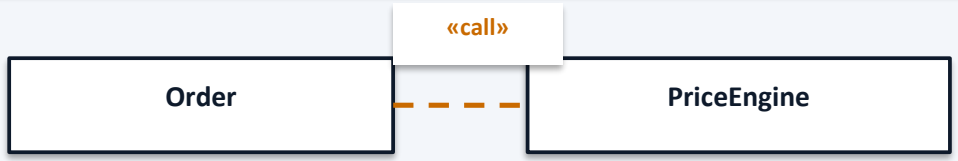
model management: requirements → design → code lineage

You may define your own stereotypes, UML allows it, but in this course an unlabelled dashed arrow means «use», and «create» or «call» earn their place only when the kind of usage is the point of the diagram.

Why anyone should care: you cannot delete dependencies, you can only choose their target

Order.java — coupled to a concrete static

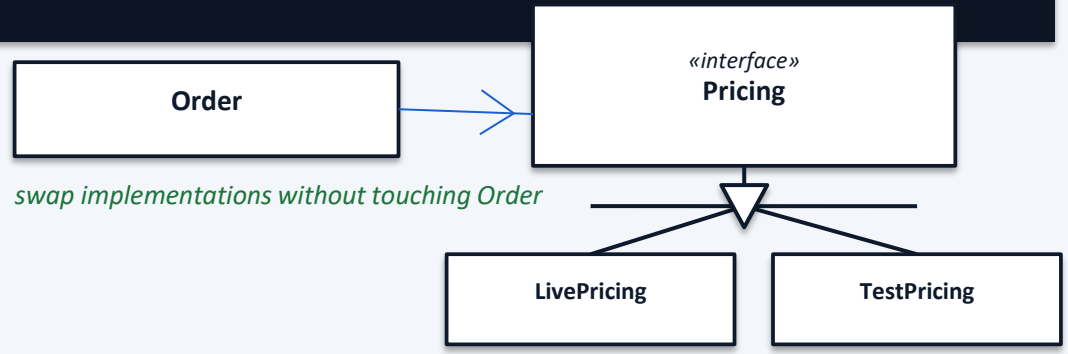
```
// before: hard-wired
public BigDecimal total() {
  return PriceEngine.total(lines);
}
```



one implementation, forever

Order.java — coupled to an interface

```
// after: aim at an abstraction
private final Pricing pricing;   // injected
public BigDecimal total() {
  return pricing.total(lines);
}
```



swap implementations without touching Order

This is what seniors mean by “low coupling” and “depend on abstractions”: the dashed arrows do not disappear, they retarget onto something that changes slowly. UML lets you see that retargeting before you code it.

Reading & drawing the diagram

The notation card: association vs dependency, side by side

	ASSOCIATION	DEPENDENCY
Glyph	**solid line**	**dashed line + open arrowhead**
Relates	classifiers; instances are links between objects	named elements: classes, interfaces, packages
Meaning	instances are linked / a reference is stored	client requires supplier's definition
Arrow rule	stick arrowhead at the navigable end (optional); × = not navigable	open arrowhead at the supplier (mandatory)
Adornments	name + reading ▸, role names, multiplicity, ownership dot	stereotype («use», «create», «call»…), name
Code signal	a field of that type	parameter, local, static call, return type, import
Breaks when	the stored reference is removed	the supplier's used definition changes
If both apply	**draw only the association**	**the dependency stays implicit**

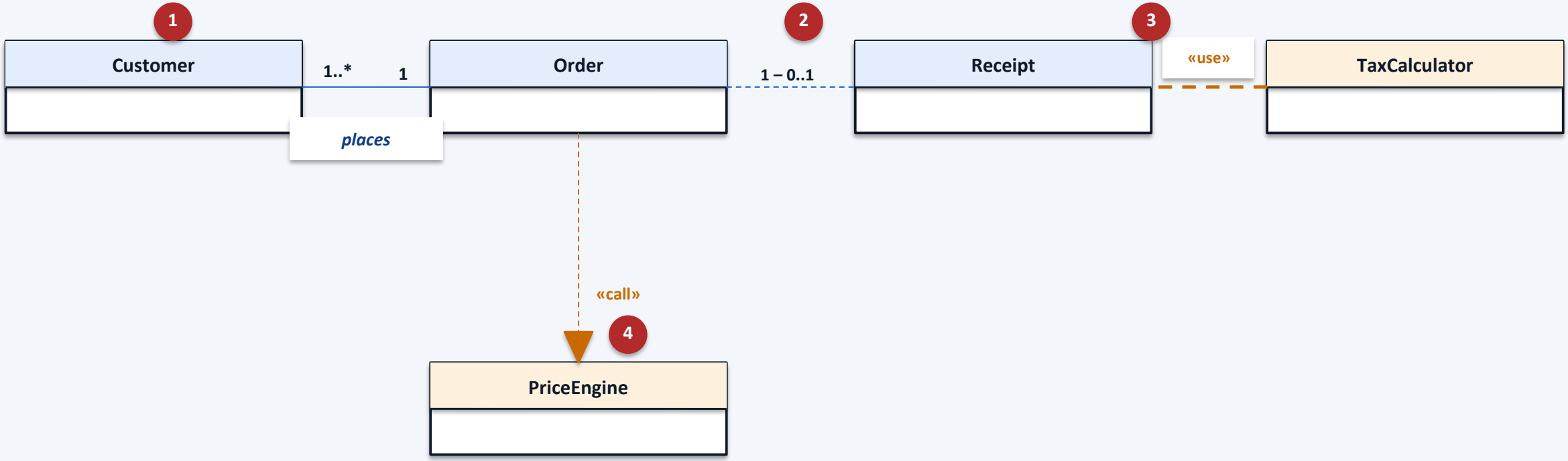


✗ as drawn
each phone owns 0 or more people; each person is owned by 1 phone.



✓ as intended
each phone is owned by 1 person; each person owns 0 or more phones.

Four mistakes are hiding in this diagram



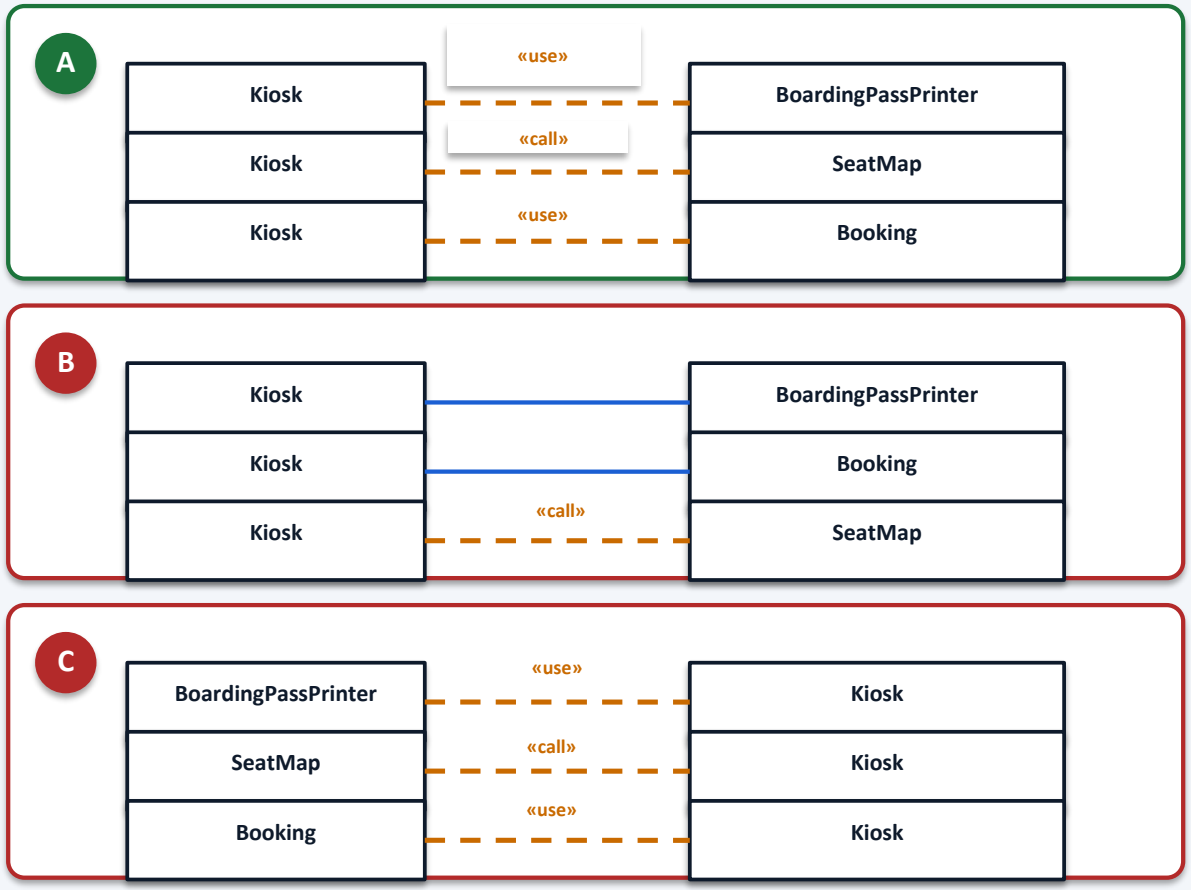
Hunt list: **(1)** multiplicities on ‘places’; **(2)** the line between Order and Receipt; **(3)** the arrow between TaxCalculator and Receipt; **(4)** the arrowhead into PriceEngine.Write your four verdicts now, the corrected diagram is the next slide, no taking it back.

Code → diagram: which of A, B, C is correct?

Kiosk.java

```
public class Kiosk {
  public BoardingPass checkIn(
          Booking b,
          BoardingPassPrinter p) {
    SeatMap.reserve(b.seat());
    return b.issue();
  }
}
```

Does Kiosk store anything? No fields at all → no association.Three names appear in the method: two as parameters/uses, one as a static call → three dependencies, all leaving Kiosk.Answer: A.



The four-question procedure: run it in this order, every time

Q1 Does an object of A keep a reference to B in a field, alive after the call returns?

→ **ASSOCIATION**
add name, multiplicity, navigability

Q2 Otherwise: does B appear in A's signature, body, static call or return type?

→ **DEPENDENCY**
point client → supplier; pick a stereotype if the kind matters

Q3 Otherwise: must instances still be paired, with data on the pair itself?

→ **LINK / ASSOCIATION CLASS**
a third class holds the pair — formal notation next lecture

Q4 None of the above?

→ **NO LINE**
the strongest model is often the emptiest diagram

Cross-check: if Q1 fired, the dependency is implied, do not add a dashed arrow. One fact, one glyph.

Modelling guidelines, and the five traps that cost marks

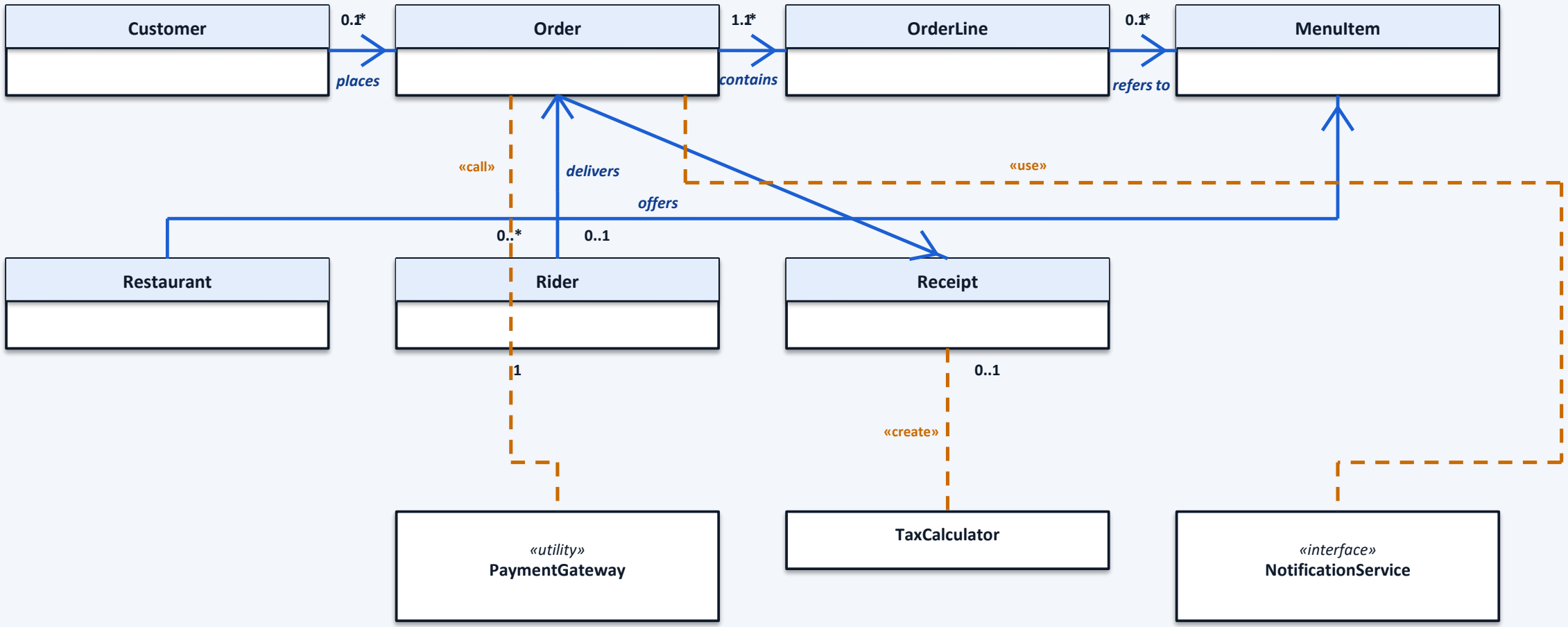
GUIDELINES (THE CRAFT)

- Name associations with verb phrases (`places, contains, delivers`); nouns are for classes.
- Take multiplicities from domain rules, not from what your current code happens to hold.
- Draw navigability only when you know it; unspecified is an honest answer.
- Omit dependencies that carry no information (language types, your own trivial helpers). A diagram is a filter, not a transcript.
- Keep one fact per glyph: the stronger relationship wins.

THE FIVE TRAPS (THE MARKS)

- **Multiplicity at the wrong end**, read it standing at the opposite class, always.
- **Dashed arrow pointing the wrong way**, client → supplier, i.e. towards the class you fear to change.
- **Arrowhead alphabet swaps**, stick / open-on-dashed / hollow triangle / ×.
- **Drawing association AND its implied dependency**, redundant noise.
- **Calling a parameter an association**, unless it gets stored (slide 8's exception).

CampusEats, complete: both glyphs on one canvas



Read the dashed arrows as sentences: **Order «call» PaymentGateway (static charge); Receipt «create» TaxCalculator (built inside print()); Order «use» NotificationService (passed into confirm()). Three debts, three stereotypes, zero stored references.**

Your turn: AirCheck

AirCheck: requirements (read them like a client, not like a coder)

1. A Passenger flies on many Flights and a Flight carries many Passengers. Each passenger-on-flight pairing holds a seat number and a checked-in flag. Neither class alone can hold that data.
2. That pairing is a Booking object. A Booking stores exactly one Passenger and exactly one Flight.
3. A Flight stores the Aircraft assigned to it. An Aircraft keeps no list of flights it has operated.
4. To finish check-in, Booking calls the static method SeatMap.reserve(seat). Booking keeps no reference to SeatMap afterwards.
5. A Kiosk prints by calling print(pass, printer), where printer is a BoardingPassPrinter handed in by the caller. The Kiosk does not store it.
6. Booking creates the BoardingPass when check-in completes, and stores the issued pass.
7. Booking holds the field ‘private FareCalculator fares;’, where FareCalculator is an interface, and calls fares.compute(this).

YOUR TASKS · 8 MINUTES

- **A.** List the classes.
- **B.** For each statement 1–7: association or dependency? Add the stereotype where one fits.
- **C.** Draw the diagram: names, multiplicities at both ends, navigability arrowheads.
- **D.** One sentence: why does statement 1 force a third class?

Trap warning

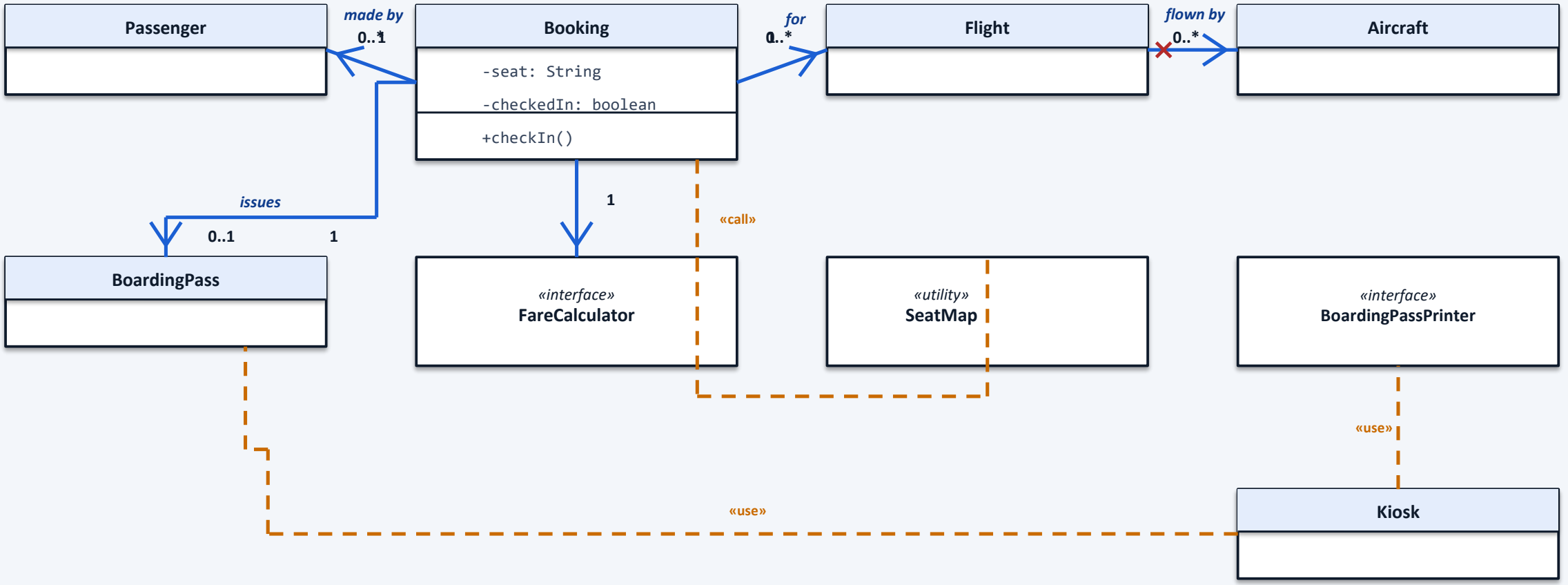
One statement is an association wearing dependency's clothes. One is a dependency wearing association's clothes. Statement 6 is both at once, decide which glyph wins.

AirCheck classified, statement by statement

#	Relationship	Glyph & adornments	Why
1	**Passenger–Flight pairing with its own data**	third class: Booking + two associations	data lives on the pair, not on either end (formal name: association class)
2	Booking → Passenger; Booking → Flight	**solid**; “1” at Passenger/Flight end, “0..*” at Booking end; arrowheads at Passenger and Flight	stored fields → structure
3	Flight → Aircraft	**solid**; “1” at Aircraft end, “0..*” at Flight end; arrowhead at Aircraft, × at Flight end	Flight stores it; Aircraft cannot navigate back
4	Booking → SeatMap	**dashed «call»**	static method, nothing stored
5	Kiosk → BoardingPassPrinter; Kiosk → BoardingPass	**dashed «use»** ×2	parameters only, never stored
6	Booking → BoardingPass	**solid**; “0..1” at pass end (none before check-in); arrowhead at pass	created AND stored → association wins; «create» stays implicit
7	Booking → FareCalculator	**solid**; arrowhead at FareCalculator	a stored field: interfaces can be association ends too; this is dependency injection

Statement 7 is the one most pairs miss: “interface” sounds like dependency, but ‘private FareCalculator fares;’ is a field, attribute visibility, so the solid line is correct.

AirCheck, drawn



Count the glyphs: five solid associations (structure), three dashed dependencies (debts), one ×, one 0..1, and not a single redundant arrow. That is a diagram worth full marks.

Summary

1. An association relates classifiers; a link is one runtime instance of it.
2. Stored reference ⇒ association. Used-only (parameter, local, static, return) ⇒ dependency.
3. Multiplicity at an end counts that end's objects per ONE object at the other end — read it standing opposite.
4. Stick arrowhead = navigable end, × = not navigable, nothing = unspecified; the arrowhead sits where the reference lives.
5. A dependency arrow runs client → supplier: “change the supplier and the client may have to change too.”
6. Association implies dependency, draw the stronger fact once; never both glyphs for one fact.

NEXT LECTURE

- whole–part: aggregation & composition
- association classes (today's Booking, formalised)
- qualified associations
- same CampusEats model, new glyphs

Next time someone's one-line change breaks your five files, you will know exactly which dashed arrow just came due, and which solid line you should have argued about in design review.